Array Methods

map, filter, reduce and who mutates your data

	map	filter	reduce
Returns	new array	new array	one value
Callback	`x => x * 2`	`x => x > 1`	`(s, x) => s + x`
`[1, 2, 3]`	`[2, 4, 6]`	`[2, 3]`	`6`

Callback gets `(item, index, array)` · start value: `reduce(fn, 0)`

MUTATES `push` `pop` `shift` `unshift` `splice` `sort` `reverse`

COPIES `map` `filter` `slice` `concat` `toSorted` `toReversed`

1 find, some, every

```
const a = [1, 2, 3];
a.find(n => n > 1)  // 2
a.find(n => n > 5)  // undefined
a.some(n => n > 2)  // true
a.every(n => n > 2) // false
```

2 sort needs a compare

```
[10, 9, 1].sort()
// [1, 10, 9]
[10, 9, 1].sort((a, b) => a - b)
// [1, 9, 10]
```

TRAP What does this log?

```
const a = [3, 1, 2];
const b = a.sort();
console.log(a === b);
```

`true`

sort() mutates and returns the **same** array. Copy first, or use **toSorted()**.

KEY TAKEAWAY

- `map/filter` — Always a new array
- `reduce` — Many items, one value
- `sort()` — Mutates: copy first

? Follow-up questions to expect

- **Which methods mutate the original?** → push, pop, shift, unshift, splice, sort, reverse.
- **What does a callback receive?** → (item, index, array). reduce: acc comes first.
- `map` **vs** `forEach`**?** → map returns a new array, forEach returns undefined.
- `reduce` **without an initial value?** → Starts from item one. Empty array: TypeError.